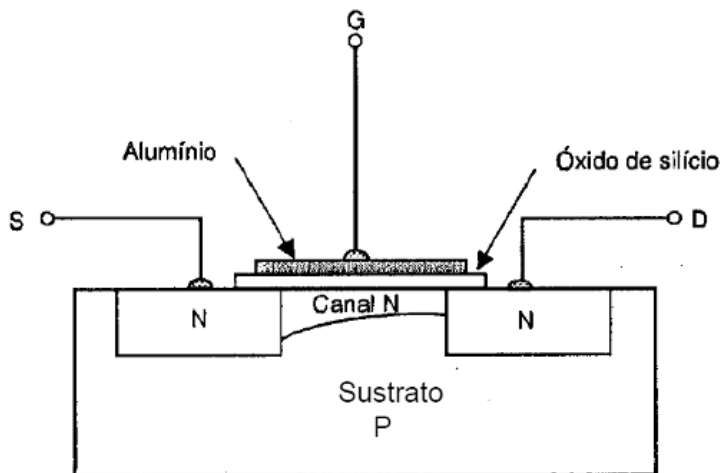




Trabajo práctico 6

Flujo de diseño analógico con CMOS

■ Autores:

- Manuel León Parfait - Leg. 406599 (Documentador)
- Marcos Raúl Gatica - Leg. 402006 (Coordinador)

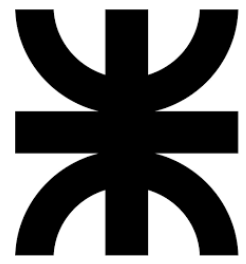
■ Curso: 3R1

■ Docentes:

- Ing. Francisco Hugo Sigampa
- Ing. Luis Alberto Guanuco

■ Asignatura: Dispositivos Electrónicos

■ Institución: Universidad Tecnológica Nacional - Facultad Regional de Córdoba.



U
T
N

F
R
C

Índice

1. RÚBRICA	1
2. INTRODUCCIÓN	2
2.1. Condiciones de trabajo y ensayo	2
2.2. Consigna del TP: Especificaciones de diseño	2
3. ACT. 1: INSTALACIÓN DE LAS HERRAMIENTAS	2
4. ACT. 2: FLUJO DE DISEÑO ANALÓGICO	3
4.1. Diseño esquemático	3
4.2. Simulación	4
4.3. Layout: Placement / Routing	5
4.4. DRC	6
4.5. LVS	7
5. CONCLUSIONES	8

1. RÚBRICA

TAREA	PUNTUACIÓN	MÁXIMO
Presentación del informe con archivos correspondientes		20 %
Comprender la estructura interna y funcionamiento del inversor CMOS		30 %
Comprender las etapas involucradas en el flujo de diseño analógico		30 %
Defensa de las conclusiones		20 %
TOTAL		100 %

2. INTRODUCCIÓN

El objetivo de este informe es comprender y aplicar el flujo de diseño analógico para circuitos integrados en tecnología CMOS, así como conocer y utilizar las herramientas *OpenSource* de diseño de C.I. para validar esquemáticos y layouts. Para ello nos valimos de familiarizarse con las reglas de diseño físico (DRC), simulaciones y verificaciones para garantizar la compatibilidad con los procesos de fabricación, e integrar el conocimiento teórico con la práctica utilizando un kit de diseño: (PDK) SKY130, junto a herramientas CAD para un diseño confiable y escalable.

2.1. Condiciones de trabajo y ensayo

El sistema operativo usado en este trabajo fue Gentoo GNU/Linux. Elegido por simple comodidad.



Figura 1: Logo Gentoo

2.2. Consigna del TP: Especificaciones de diseño

El inversor CMOS está diseñado para cumplir:

- Tecnología SKY130 (130 nm CMOS)
- Voltaje de alimentación $V_{DD} = 1,8V$
- Especificaciones de diseño para PMOS:
 - $L = 0,15$ (Longitud del canal del transistor en micrómetros).
 - $W_p = 2,1$ (Ancho del canal del transistor en micrómetros).
 - $nf = 1$ (Número de fingers).
 - $mult = 1$ (Multiplicidad).
 - model: $pfet_01v8$
- Especificaciones de diseño para NMOS:
 - $L = 0,15$ (Longitud del canal del transistor en micrómetros).
 - $W_p = 1,05$ (Ancho del canal del transistor en micrómetros).
 - $nf = 1$ (Número de fingers).
 - $mult = 1$ (Multiplicidad).
 - model: $nfet_01v8$
- Tiempo de transición objetivo: $< 50ns$ (subida y bajada)

3. ACT. 1: INSTALACIÓN DE LAS HERRAMIENTAS

Para el diseño y simulación del inversor CMOS, se necesitaron de las siguientes herramientas *Opensource* compatibles con el PDK SK130:

- **Xschem:** diseño y simulación de esquemáticos.
- **Ngspice:** modelos spice para la simulación eléctrica en el entorno Xschem.
- **Magic:** ruteo, layout y verificación del diseño físico (DRC).
- **Netgen:** comparación LVS (Layout versus Schematic).

Notas del documentador: complicaciones [X]

”En un principio, nos habíamos confiado en que los simuladores nombrados ya presentes en los repositorios de la distribución nos podrían servir. Fue cuando quisimos usar Xschem que nos dimos cuenta que faltaban los componentes de PDK SKY130, así que hicimos una adaptación del script *iic-osic-setup.sh* y *iic-init.sh* para usar el equipo completo.”

4. ACT. 2: FLUJO DE DISEÑO ANALÓGICO

4.1. Diseño esquemático

Se describe a continuación el procedimiento para desarrollar el inversor CMOS.

Flujo de trabajo: Diseño esquemático

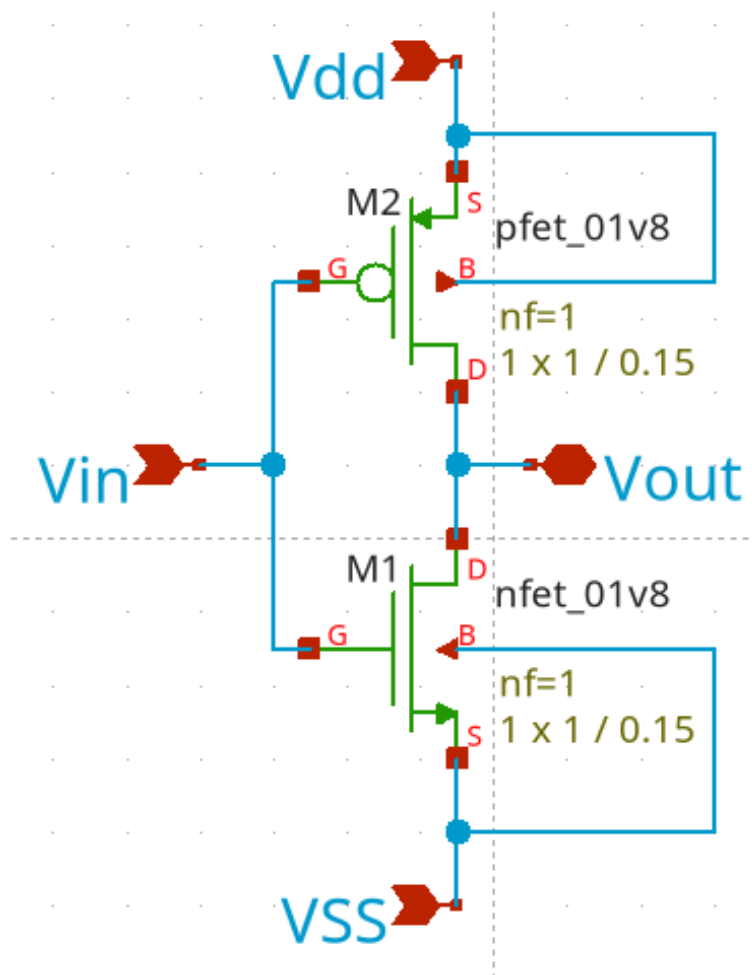
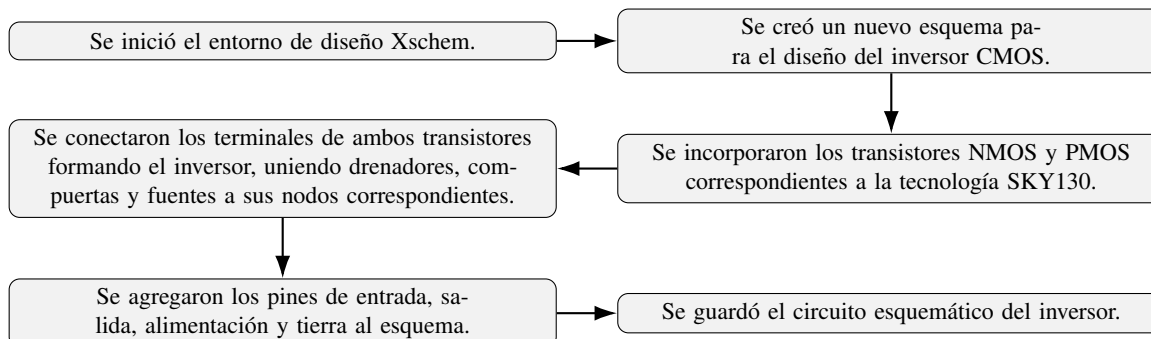
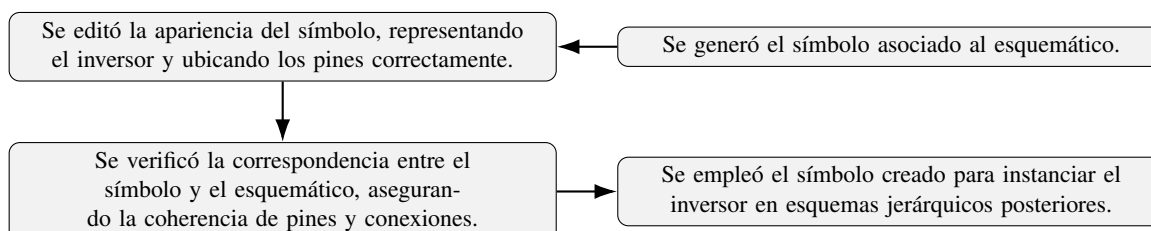


Figura 2: Diseño en Xschem



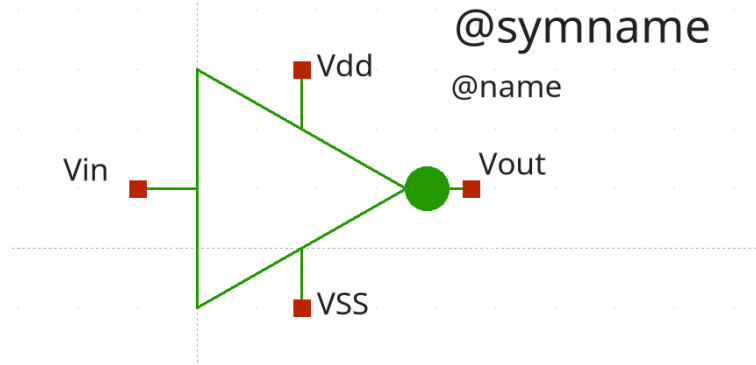


Figura 3: Simbolo del inversor

4.2. Simulación

El objetivo de este punto es usar Ngspice para verificar que la transferencia de tensión de entrada a salida sea la esperada: que se invierta.

Flujo de trabajo: Preparación para la simulación

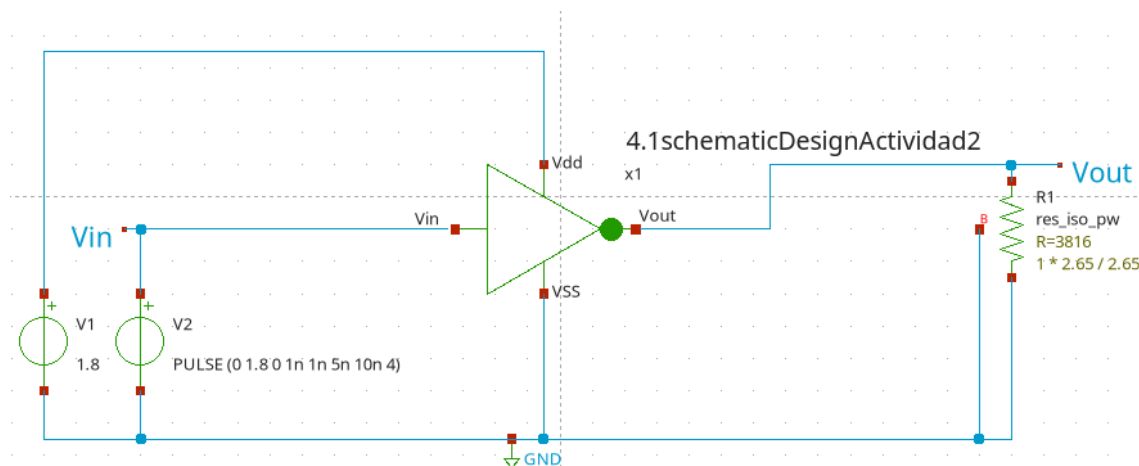
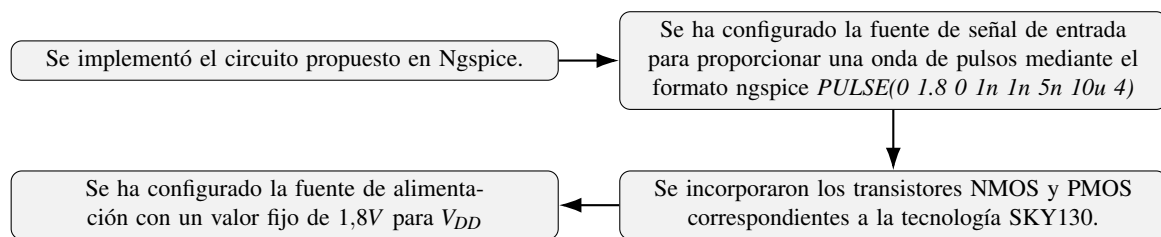


Figura 4: Circuito para simulación

Se incluyó los archivos corner tt y modelos, apuntando a la variable de entorno \$PDK y al archivo sky130_fd_sc_hd.spice

Se incluyó el archivo *s1* con la configuración del tipo de simulación.

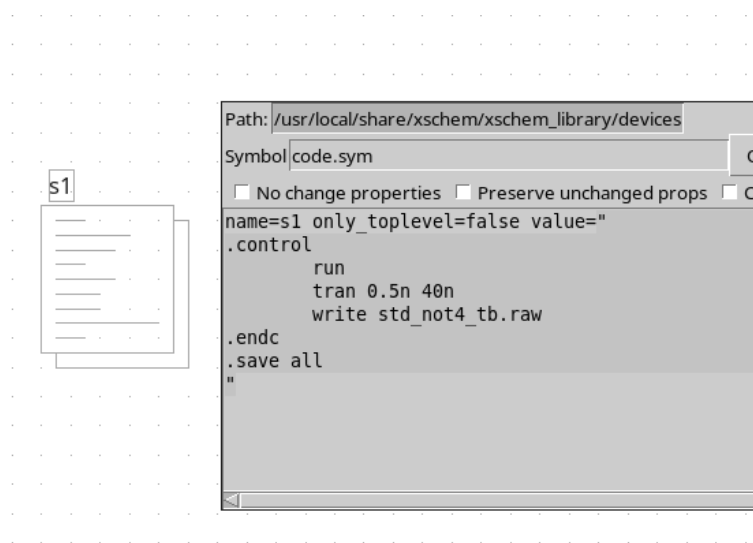


Figura 5: Archivo de configuración del simulador

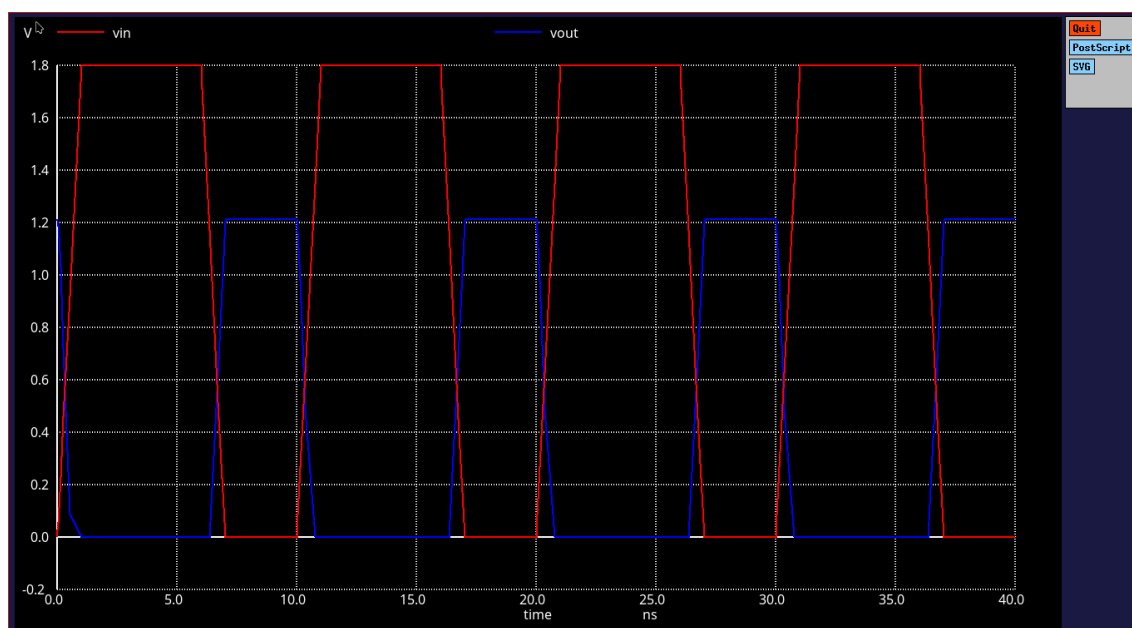


Figura 6: Gráfico obtenido en la simulación por NgSpice

Notése que cuando en la entrada está en un pico (curva V_{IN} roja), la salida del inversor (curva V_{OUT} azul) hay un 0 y viceversa. Podemos concluir que en este punto el diseño del amplificador funciona.

4.3. Layout: Placement / Routing

En este apartado se realizará el diseño físico de ambos transistores siguiendo las reglas del PDK SKY130.

PLACEMENT

Tras importar el modelo spice, el simulador Magic "acomodó" automáticamente los componentes, sin embargo se decidió borrar las capas generadas para ir cargando los transistores manualmente, acomodarlos y conectarlos, de manera que el flujo de trabajo realizado fue:

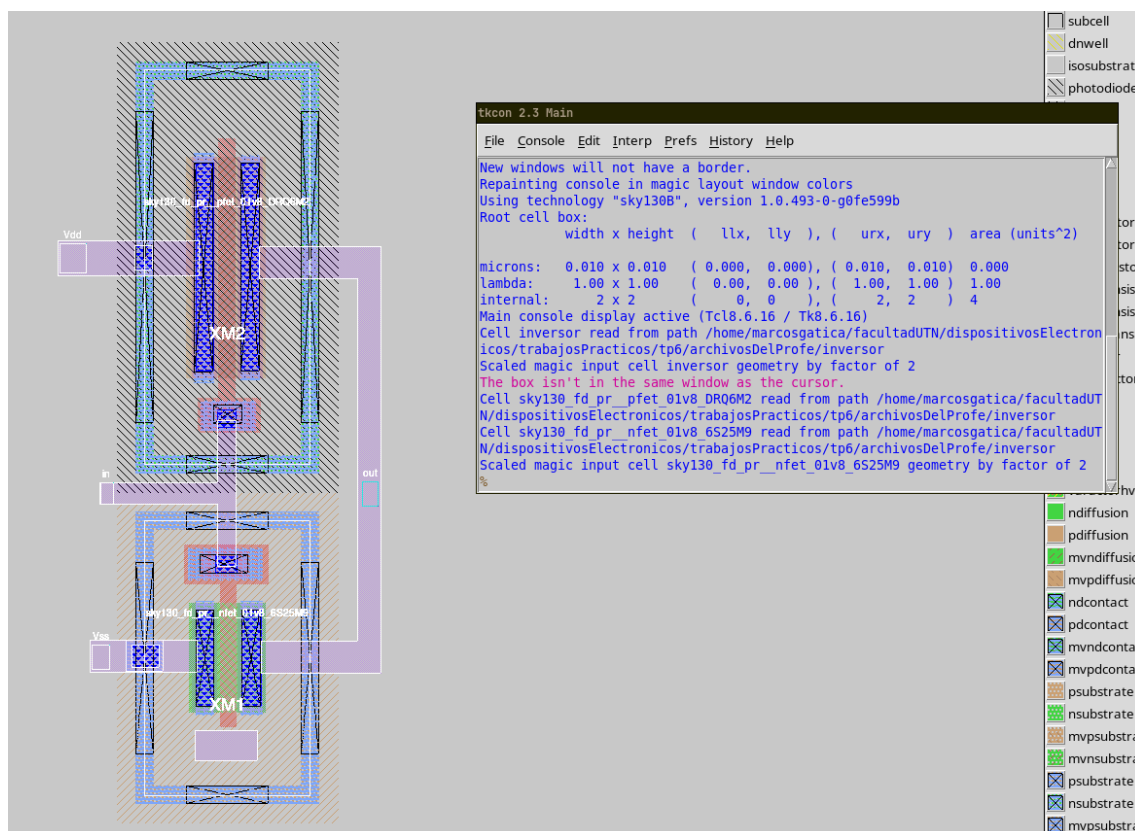
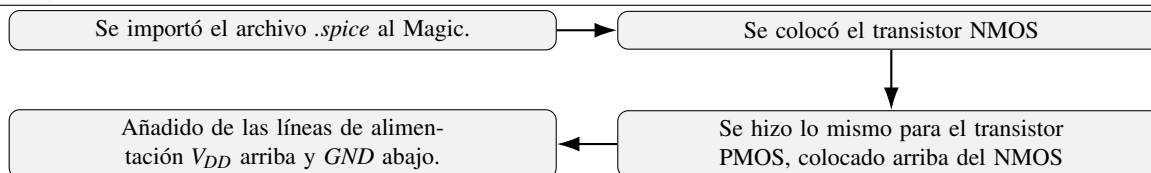


Figura 7: Placement del inversor en Magic

ROUTING

Los puntos a cumplir en el *routing* son:

- Conectar las compuertas entre sí con polisilicio para formar las entradas comunes.
- Unir los drenadores de PMOS y NMOS con metal para crear la salida.
- Trazar rutas de metal para las líneas de alimentación V_{DD} y GND , conectando las fuentes de los transistores.

4.4. DRC

El DRC son las verificaciones de Magic para las reglas de diseño. Permite detectar capas sobrepuestas, distancias insuficientes y/o contactos incorrectos.

En la consola de Magic se puede activar las verificaciones con:

```
% drc check
```

Notas del documentador: complicaciones X

”Si bien la guía para desarrollar el trabajo menciona que el comando:”

```
% drc show
```

”permite ver el resumen de los análisis de DRC, en el simulador Magic que instalamos usando la guía recomendada nos arroja error al no reconocer a show como argumento para el comando drc. Sin embargo, concluimos que el diseño está libre de errores al ver el box-check de Magic: si DRC=0, el diseño tiene 0 violaciones de reglas.”



Figura 8: DRC check

”Alternativamente se puede visualizar en la terminal un reporte de DRC en el menú:”

DRC → DRC report

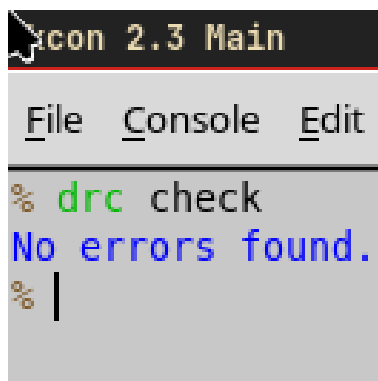


Figura 9: Salida de DRC report

4.5. LVS

LVS es la verificación de correspondencia entre layout y esquemático.

I. 1) Generar netlist

- En Magic se extrae el netlist para LVS con:

```
% extract all  
% ext2spice -lvs
```

- Se exporta además el netlist del esquemático desde Xschem con:

```
$ xschem --export ngspice schematic.sch -o schematic.spice
```

II. Preparación de archivos para Netgen

- Ambos archivos extraídos (los netlist) están en *.spice*

III. Ejecución de Netgen

```
Cell pin lists are equivalent.  
Device classes sky130_fd_pr__pfet_01v8 and sky130_fd_pr__pfet_01v8 are equivalent.  
Flattening unmatched subcell 41schematicDesignActividad2 in circuit 42simulationSchematic.spice (0)(1 instance)  
  
Cells have no pins: pin matching not needed.  
Device classes 42simulationSchematic.spice and inversorLvsLayout.spice are equivalent.  
  
Final result: Verify: cell inversorLvsLayout.spice has no elements and/or nodes. Not checked.
```

Figura 10: Primera comprobación de LVS

IV. Análisis

La ejecución de Netgen devolvió que la celda de nuestro archivo *inversorLvsLayout.spice* no posee elementos y/o nodos. El error que se cometió fue que no se terminó de escribir en disco el mencionado archivo, al abrirlo daba error. Por lo que se recuperó el archivo, se comprobó el DRC de antes y se intentó de nuevo. Devolviendo:

```
Cell pin lists are equivalent.  
Device classes inverter and inverter are equivalent.
```

```
Final result: Circuits match uniquely.
```

```
+
```

Figura 11: Segunda comprobación de LVS

Se concluyó que al ver "Resultado final: Los circuitos coinciden de forma única.", el circuito pasa la comprobación de LVS.

5. CONCLUSIONES

***Desafíos:** diseño y simulaciones circuito analógico CMOS*

3
